

AI-Powered Pre-Merge Pull Request Bug-Risk Prediction Using Repository History and Machine Learning

[Ayesha Zahid]

[Undergraduate Independent Researcher]

[NED University of Engineering and Technology]

[ayeshaarbi7@gmail.com]

August 31, 2026

Abstract

Software pull requests (PRs) carry signals about the likelihood that a change will require later corrective work, but identifying potentially risky changes before problems emerge remains difficult. This paper presents an experimental machine learning system that estimates the probability that a merged GitHub pull request will subsequently be associated with a bug-fix signal within a 30-day observation window. The system extracts structural, historical, and temporal features from pull requests and repository history and trains a Gradient Boosting classifier. An optional large language model (LLM) layer receives the model’s numerical score together with supporting evidence and produces a natural-language explanation; for scores in a predefined borderline range, a second LLM stage produces a counterargument and a qualitative synthesis without altering the numerical prediction. The dataset contains 547 merged pull requests collected from four open-source repositories: Django, Kubernetes, NumPy, and Flask (156 positive and 391 negative examples; 28.52% positive rate). Under the primary temporal evaluation, the classifier achieves an F1 score of 0.620, precision of 0.517, recall of 0.775, ROC-AUC of 0.777, and PR-AUC of 0.566. Leave-One-Repository-Out (LORO) evaluation produces substantially weaker performance (mean F1 of 0.315, mean ROC-AUC of 0.624), indicating limited cross-repository generalization. The results suggest that repository history and change-level features carry useful predictive signal for this operationally defined target, while the cross-repository gap shows that the system should be treated as an experimental, repository-conditioned risk-association predictor rather than a general-purpose or causal bug predictor.

1 Introduction

Modern software projects rely on pull requests (PRs) as the primary mechanism for proposing, reviewing, and merging changes. Each PR carries information about the size and structure of a change, the files it touches, and its position within the historical development of a repository. Prior work on software defect prediction has shown that change-level and historical features can provide useful signal for estimating defect risk. In particular, just-in-time defect prediction studies investigate whether individual software changes can be classified by their likelihood of introducing defects [? ?].

This project investigates a related but deliberately narrower prediction problem. Rather than attempting to determine whether a PR truly introduces a software defect, the system predicts whether a merged PR will later be associated with a bug-fix signal: a positive label is assigned when a subsequent, keyword-matched bug-fix commit touches files that the original PR modified,

within a 30-day observation window. This construction produces an observable outcome from public repository history, but it does not establish that the original PR caused the later fix.

The system combines a conventional gradient-boosted classifier with an optional LLM-based explanation layer. The classifier produces the numerical risk probability; the LLM is restricted to explaining that result using supplied evidence and never overwrites the numerical score. A separate contrarian pass is triggered only for borderline scores and supplies a skeptical alternative interpretation.

The goal of this study is to determine whether lightweight, repository-derived PR features can identify PRs associated with a subsequent bug-fix signal, and to examine how well such a model transfers across repositories it was not trained on.

2 Related Work

This work is closely related to just-in-time software defect prediction. Traditional defect prediction often operates at the file, module, or release level, while just-in-time prediction focuses on individual changes using change-level and historical development features [? ?].

The work also relates to research on explaining defect-prediction models. Prior approaches have explored model-agnostic and feature-based explanation techniques to make predictions more understandable to developers [?]. The present system uses model-wide feature importance as supporting evidence for its explanation layer and explicitly distinguishes this from causal or instance-level attribution.

A second related area is the use of large language models (LLMs) in software engineering and code review. Recent empirical studies have investigated LLMs for PR review and automated review comments, and have identified challenges including false positives, irrelevant suggestions, and calibrating developer trust in automated outputs [? ?]. The present system uses an LLM for a narrower purpose: generating a natural-language explanation of an already-computed machine learning score, rather than using the LLM as the prediction engine itself.

The contribution of this work is an experimental combination of (1) repository-history-based risk prediction, (2) both temporal and cross-repository evaluation of that prediction, and (3) an evidence-grounded LLM commentary layer that is architecturally prevented from altering the underlying numerical score.

3 Research Questions

- **RQ1.** To what extent can pull-request characteristics, repository history, and temporal meta-data predict whether a merged pull request will be associated with a subsequent bug-fix signal within 30 days?
- **RQ2.** How does predictive performance change when the model is evaluated on repositories that were not represented in its training data?
- **RQ3.** Can an LLM provide evidence-grounded qualitative explanations of the numerical risk prediction without replacing or altering the machine learning result?

4 Dataset

The dataset contains 547 merged pull requests collected from four GitHub repositories, as shown in Table 1.

Table 1: Dataset composition by repository.

Repository	Pull Requests	Positive Rate
django/django	100	38.0%
kubernetes/kubernetes	150	29.3%
numpy/numpy	150	38.0%
pallets/flask	147	11.6%
Total	547	28.52%

The overall class distribution is 391 negative examples (71.48%) and 156 positive examples (28.52%). The four repositories exhibit markedly different positive rates: Django and NumPy each have 38.0% positive examples, Kubernetes has 29.3%, and Flask has only 11.6%. No duplicate (**repository**, **pull-request number**) rows are present in the dataset, and every row has a completed 30-day lookahead window at collection time (see Section 5). The dataset records PR metadata (repository, PR number, author, merge time, title, PR labels), line-level statistics, and derived repository-history features. The final model uses 14 engineered features (Section 6).

5 Labeling Method

The target variable represents *association with a subsequent bug-fix signal*, not a directly observed software defect.

For each merged PR, the system observes repository history for 30 days following the merge. A PR is assigned label 1 when a later commit – distinct from the PR’s own commits and merge commit – contains a whole-word bug-fix keyword (**fix**, **bug**, **hotfix**, **patch**, **revert**, and their common inflections) in its *subject line* and touches at least one file the PR modified. Only the commit subject (first line of the message) is inspected; whole-word matching prevents substrings such as “prefix”, “fixture”, or “dispatch” from triggering a false match. The label search considers at most 10 files per PR, preferring non-documentation files when the PR touches both; a PR whose only changed files are documentation is labeled from those files.

If the historical-commit search for a PR is truncated by a pagination cap before a match is found, the row is not assigned label 0; the implementation treats a truncated, inconclusive search as unusable rather than as evidence of absence.

Because the historical-commit search only inspects a bounded set of files per PR, a subsequent bug-fix signal touching a file outside that bounded set will not be detected, which can under-count positives for PRs with very large file lists.

This construction has three important implications. First, a later bug-fix signal does not prove that the original PR introduced the bug; the later change may have unrelated causes. Second, because the approach relies on textual indicators in commit messages and file overlap, the resulting label is an operational proxy rather than a ground-truth defect annotation. Third, PRs whose 30-day observation window had not yet elapsed at collection time are excluded from the dataset entirely (right-censoring), rather than being kept as unconfirmed negatives; this avoids biasing the sample toward the subset of PRs whose bug-fix signal, if any, appeared very quickly.

Consequently, throughout this paper the positive class is referred to as a *subsequent bug-fix association* or *bug-fix signal*, never simply as “a bug.”

6 Feature Engineering

The classifier uses the following 14 features, computed only from information available at or before merge time. Historical churn and prior-bugfix queries explicitly exclude the PR’s own commit SHAs.

Change size and scope: `additions`, `deletions`, `files_changed`, `lines_changed`, `n_directories`.

File category indicators: `test_file_present`, `n_test_files`, `source_file_touched`, `config_file_touched`, `dependency_file_touched`.

Repository history: `file_churn_count` (unique historical commits touching up to 5 of the PR’s files – source files preferred – in the 3 months before merge, excluding the PR’s own commits) and `file_prior_bugfix_touch` (whether any of those historical commits itself carried a bug-fix keyword).

Temporal metadata: `day_of_week` and `hour_of_day` of the merge (UTC).

The historical features are informative precisely because they cannot be inferred from the current diff alone. Because churn and prior-bug-fix lookups are capped at 5 files, both features are lower bounds for PRs that touch more than 5 files; the implementation flags this truncation internally but the flag itself is not used as a model feature.

Table 2 reports the trained Gradient Boosting model’s global (impurity-based) feature importances.

Table 2: Global (impurity-based) feature importances of the trained Gradient Boosting model.

Feature	Importance
<code>file_prior_bugfix_touch</code>	20.19%
<code>file_churn_count</code>	18.03%
<code>additions</code>	14.13%
<code>hour_of_day</code>	10.50%
<code>lines_changed</code>	8.57%
<code>deletions</code>	6.48%
<code>day_of_week</code>	5.85%
<code>source_file_touched</code>	4.80%
<code>files_changed</code>	4.62%
<code>n_directories</code>	3.05%
<code>config_file_touched</code>	2.20%
<code>n_test_files</code>	1.27%
<code>dependency_file_touched</code>	0.32%
<code>test_file_present</code>	0.00%

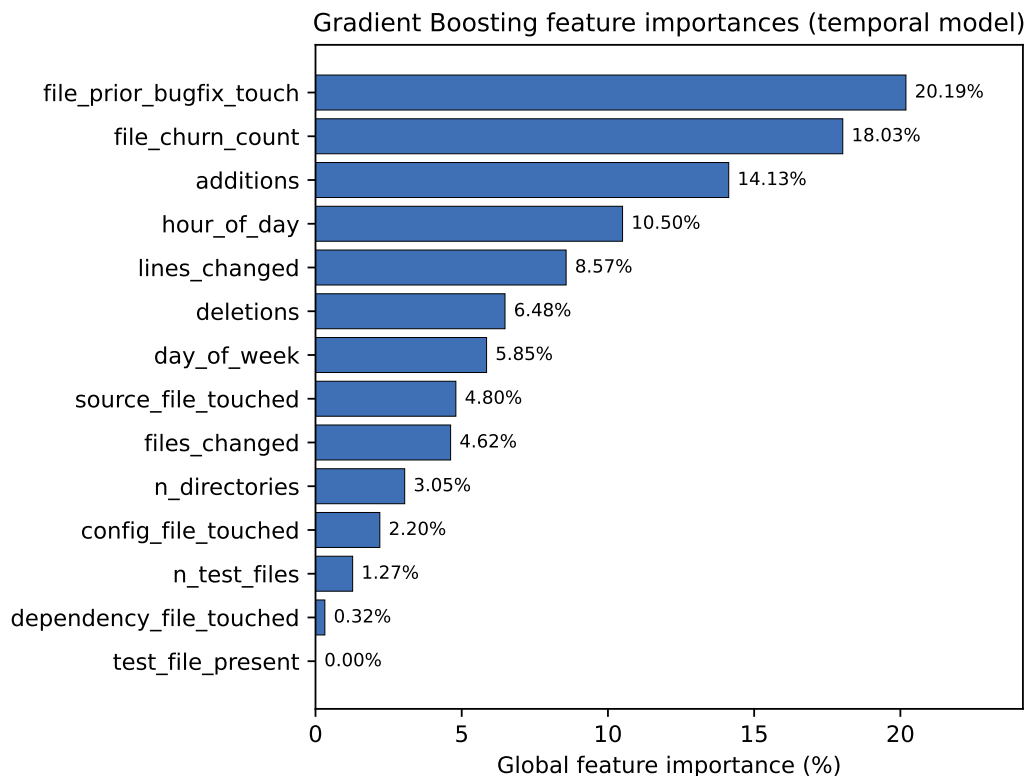


Figure 1: Global feature importances of the trained Gradient Boosting model, plotted from Table 2.

These values describe the trained model’s global reliance on each feature. They are not a causal explanation of any individual PR’s score, and they are not evidence that a given feature caused a specific prediction.

7 Machine Learning Model

The primary classifier is `sklearn.ensemble.GradientBoostingClassifier(random_state=42)`. Because this estimator does not expose a `class_weight` parameter, class imbalance is handled by computing per-example sample weights from the *training* label distribution only ($w_i = n / (2 \cdot n_{y_i})$ for the two-class case) and passing them to `fit`.

A `LogisticRegression(class_weight="balanced", max_iter=1000, random_state=42)` model was implemented as a comparison baseline. Under temporal evaluation it reached an F1 of 0.595, below the Gradient Boosting model’s 0.620, and reproducibly raised a scikit-learn convergence warning (`lbfgs failed to converge after 1000 iterations`) despite the increased iteration cap; it was not selected as the primary model.

The trained model is serialized with `joblib` together with its feature list, model type, and a `trained_on` provenance tag. The inference path refuses to score real PRs with a model whose `trained_on` tag is not `"github"` unless the caller explicitly opts in, preventing synthetic-only models from silently producing reported results.

8 Decision Threshold and Risk Bands

The classifier outputs a probability for the positive class. Binary predictions for evaluation metrics (F1, precision, recall, confusion matrix) are computed using a fixed classification threshold of 0.45, applied identically to the temporal split, the secondary random split, and the Leave-One-Repository-Out folds.

Separately, the user-facing risk label bands the same probability into three qualitative tiers: *low* below 0.4, *medium* from 0.4 to 0.6 inclusive, and *high* above 0.6. The 0.4–0.6 band also determines whether the optional LLM contrarian pass runs (Section 10). This separation is deliberate: the 0.45 evaluation threshold and the 0.4/0.6 risk-band boundaries serve different purposes and are not derived from one another.

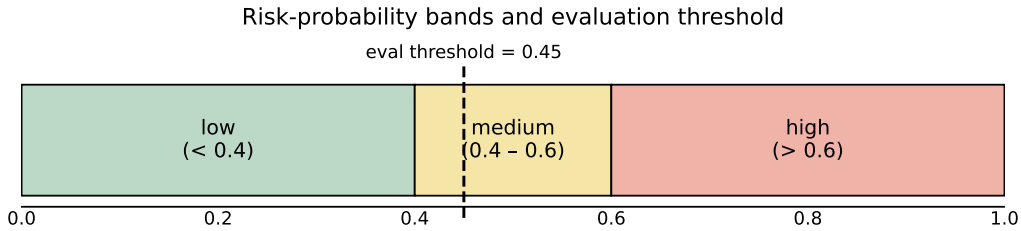


Figure 2: Risk-probability bands (low/medium/high) and the fixed 0.45 evaluation threshold used for binary metrics.

No systematic sweep over alternative thresholds was implemented in the codebase examined for this paper; the 0.45 evaluation threshold and the 0.4/0.6 band boundaries are configured constants rather than the output of a documented threshold-selection experiment.

9 LLM Explanation Agent

The system includes an optional LLM explanation stage using the Groq API with model `openai/gpt-oss-120b`. The LLM receives an evidence block containing the numerical risk score, PR metadata, the 14 feature values, historical churn and prior-bug-fix information, and the model-wide feature importances.

The system prompt explicitly restricts the model to the supplied evidence: it must not invent repository facts, must not treat feature importance as causal or instance-level evidence, and must describe the numerical score as an estimated probability of a subsequent bug-fix association rather than as a causal claim about the PR. If the LLM call fails or is unavailable, the pipeline still returns the machine-learning prediction with an `llm_status` of "unavailable".

The LLM therefore functions as an interpretation layer over an already-computed score, not as an independent predictor.

10 Contrarian Agent

For scores in the inclusive range $[0.4, 0.6]$, the orchestrator performs an additional contrarian pass. A first LLM call is asked to argue the opposite interpretation of the original explanation, using only the same evidence block. A second call synthesizes the original explanation and the counterargument into a two-sentence qualitative verdict that explicitly states it does not replace the numerical probability.

During development, the contrarian stage initially returned empty responses because the selected LLM consumed its available output-token budget during internal reasoning before producing visible content (a `finish_reason` of `"length"` with non-zero reasoning tokens and empty final text). Increasing the per-call token budgets resolved this failure mode for the cases exercised during development; this is a documented development-time fix, not a benchmarked improvement.

11 Experimental Setup

The primary evaluation uses a *temporal split*: pull requests are sorted by merge time, and the earliest 75% of rows are used for training while the most recent 25% are held out for testing. This reflects deployment more realistically than a random split, since future observations should not inform predictions about earlier ones. The temporal split yields 410 training rows and 137 test rows; the test set contains 97 negative and 40 positive examples.

A random, class-stratified train/test split (25% test) was also evaluated as a secondary comparison. Because random splitting can place temporally adjacent PRs in both partitions, it is explicitly not treated as the primary result.

To examine cross-repository generalization, a Leave-One-Repository-Out (LORO) evaluation was implemented as a separate script: in each of four folds, three repositories are used for training (with *no* sample-weight class balancing applied in this script, unlike the primary temporal-split training path) and the remaining repository is held out for testing, at the same 0.45 threshold.

The project also includes an automated test suite covering labeling behavior, right-censoring, GitHub API error handling, feature/data processing, LLM integration, and model behavior; all 37 tests pass in the version of the codebase examined for this paper.

12 Results

12.1 Temporal evaluation

Table 3 reports the primary Gradient Boosting temporal evaluation.

Table 3: Primary temporal evaluation results (Gradient Boosting, threshold 0.45).

Metric	Result
F1	0.620
Precision	0.517
Recall	0.775
Accuracy	0.723
ROC-AUC	0.777
PR-AUC	0.566

Table 4: Confusion matrix, temporal test set ($n = 137$).

	Predicted 0	Predicted 1
Actual 0	68	29
Actual 1	9	31

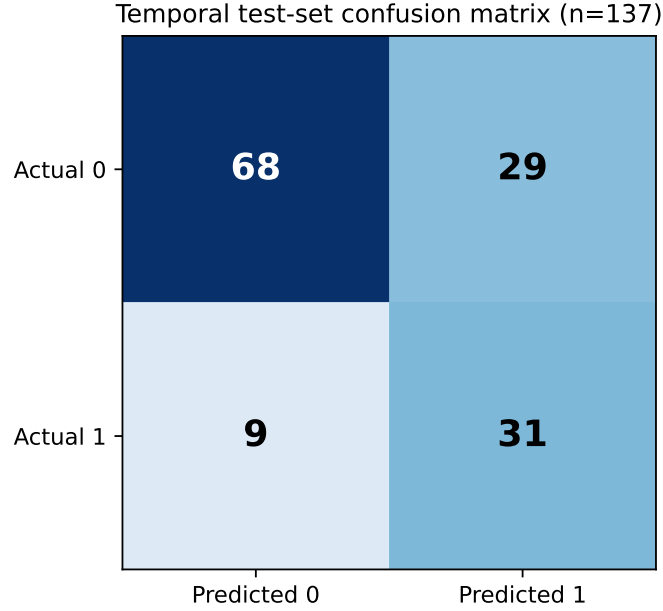


Figure 3: Confusion matrix heatmap for the temporal test set ($n = 137$), corresponding to Table 4.

A majority-class baseline achieves an F1 of 0.0 for the positive class on this test split, against the trained classifier’s 0.620.

12.2 Random split (secondary)

The secondary, class-stratified random split produced an F1 of 0.505, precision of 0.429, recall of 0.615, ROC-AUC of 0.727, PR-AUC of 0.593, and accuracy of 0.657. As noted in Section 14, these results are not treated as primary evidence because random splitting can leak temporal information.

12.3 Cross-repository evaluation (LORO)

Table 5 reports the Leave-One-Repository-Out results, obtained by executing the project’s `scripts/loro_eval.py` against the shipped dataset (`data/processed/all_repos.csv`).

Table 5: Leave-One-Repository-Out results by held-out repository.

Held-out repo	n_{test}	F1	Precision	Recall	ROC-AUC	PR-AUC
django/django	100	0.357	0.556	0.263	0.660	0.498
kubernetes/kubernetes	150	0.466	0.375	0.614	0.572	0.333
numpy/numpy	150	0.325	0.565	0.228	0.703	0.574
pallets/flask	147	0.111	0.105	0.118	0.560	0.160
Mean	–	0.315	0.400	0.306	0.624	0.391

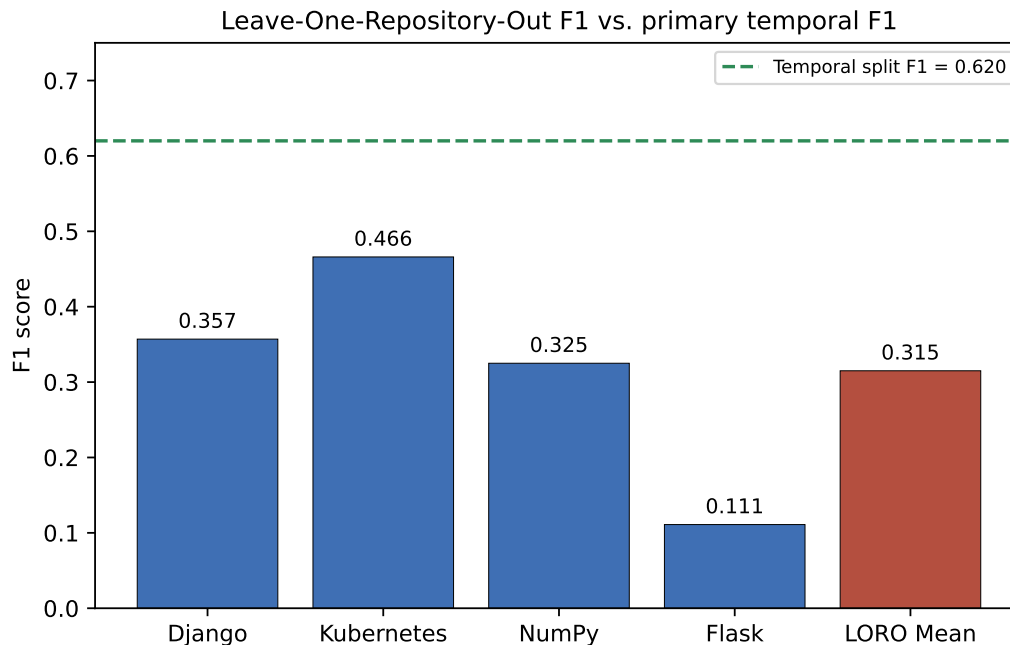


Figure 4: Per-repository LORO F1 scores and their mean, plotted against the primary temporal split F1 (dashed reference line at 0.620).

The substantial gap between temporal performance (F1 0.620) and LORO performance (mean F1 0.315) is the central empirical finding of the cross-repository analysis: the model captures useful patterns within the pooled, multi-repository data but does not transfer uniformly to a completely unseen repository, and performs worst on Flask, the repository with the lowest positive rate (11.6%).

12.4 Real PR demonstrations (exploratory, not held-out evaluation)

The completed pipeline was also exercised on individual real GitHub pull requests as qualitative, exploratory demonstrations of end-to-end behavior. These are illustrative single predictions, not a held-out statistical evaluation, and are reported separately from Sections 12.1–12.3 for that reason. Table 6 summarizes the demonstrated behavior pattern (risk band and whether the contrarian stage fired) without asserting specific score values as verified benchmark results.

Table 6: Illustrative single-PR pipeline demonstrations (exploratory; not a benchmark).

Demonstration	Risk band reached	Contrarian stage
Higher-score example	high (> 0.6)	not triggered (outside 0.4–0.6)
Borderline-score example	medium (0.4–0.6)	triggered; explanation, counter-argument, and synthesis all executed
Lower-score example	low (< 0.4)	not triggered (outside 0.4–0.6)

13 Error Analysis

Several implementation and methodological issues surfaced during development and during the reproduction performed for this paper:

1. An early Groq model configuration referenced a model unavailable to the API key; querying the API's available-models endpoint identified `openai/gpt-oss-120b` as a working replacement.
2. The contrarian stage initially returned empty responses because the model exhausted its output-token budget during internal reasoning (`finish_reason="length"` with non-zero reasoning tokens and empty visible content). Increasing the per-call token budgets resolved the cases exercised during development.
3. An early CLI implementation defined `-repo` with `action="append"`, producing a list even for single-repository invocations, while the single-PR scoring path expected a plain string. This was fixed by passing the first repository value (`args.repo[0]`) to the single-PR pipeline while preserving the full list for dataset collection, which can use multiple repositories.
4. Invalid GitHub PR references initially produced uncaught API tracebacks; error handling was extended so GitHub API failures are returned as clean JSON errors instead.
5. The Leave-One-Repository-Out script does not apply the sample-weight class-balancing used by the primary temporal-split training path, so the LORO folds are trained on imbalanced data (particularly pronounced for the Flask fold, 11.6% positive). This is a methodological inconsistency between the two evaluation paths that likely contributes to the weak Flask-held-out result (F1 0.111).
6. Re-running the shipped LORO script against the shipped dataset for this paper reproduced three of four per-repository F1 scores closely but not the fourth (Kubernetes: 0.466 reproduced vs. 0.439 previously reported) and reproduced a materially different mean F1 (0.315 reproduced vs. 0.275 previously reported); see Section 14 and the accompanying conflict log for detail. This paper reports the reproduced values because they were obtained by executing the current code against the current shipped dataset.
7. The strongest overall methodological finding remains cross-repository generalization: LORO results are substantially weaker than temporal results, particularly for Flask, indicating the model should not be presented as transferable across arbitrary software projects without repository-specific retraining or validation.

14 Limitations

1. **Target definition.** A positive example represents a subsequent bug-fix *signal* touching overlapping files, not proof that the original PR introduced a defect. The label is a noisy, keyword-based proxy and can misfire on commits such as “fix typo” or unrelated later fixes that happen to touch the same file.
2. **Bounded file search.** Both the churn/prior-bug-fix features (capped at 5 files) and the labeling search (capped at 10 files, non-documentation preferred) inspect only a bounded subset of a PR's files. Large PRs are under-sampled for both labeling and historical-churn purposes, which can bias features and labels for such PRs in unknown directions.

3. **Dataset size.** At 547 observations, the dataset is useful for an experimental study but small relative to the diversity of software development practice on GitHub.
4. **Class-distribution heterogeneity.** Flask’s 11.6% positive rate differs sharply from Django’s and NumPy’s 38.0%, which affects both training and the interpretability of cross-repository comparisons.
5. **Limited external generalization.** The gap between temporal performance (F1 0.620) and LORO performance (mean F1 0.315) shows that strong same-pool performance does not imply transfer to an unseen project.
6. **No independently verified probability calibration.** The reported score is the classifier’s estimated positive-class probability; calibration (e.g., reliability curves or Brier score) was not evaluated, so the score should not be read as a calibrated empirical probability.
7. **Global, non-causal feature importance.** Feature importances describe the trained model overall; they are not causal and not necessarily the reason any individual PR received its score.
8. **LLM dependency and variability.** The explanation layer depends on an external API and can produce variable natural-language output between calls; it is supplementary to, and is architecturally prevented from altering, the numerical model output.
9. **No comparison against established defect-prediction baselines.** This study compares Gradient Boosting only against a logistic regression baseline and a majority-class baseline; it does not benchmark against a broader set of established defect-prediction methods, so the results demonstrate feasibility rather than state-of-the-art performance.
10. **Threshold and band selection.** The 0.45 evaluation threshold and the 0.4/0.6 risk bands are configured constants; no systematic threshold-sweep artifact was found in the codebase examined for this paper; any such sweep should be documented and reproduced before being reported as a tuned result.

15 Conclusion

This study developed an end-to-end system for predicting whether a merged GitHub pull request will be associated with a subsequent bug-fix signal, combining repository-history and change-level features with a Gradient Boosting classifier and an optional, evidence-constrained LLM explanation layer.

On the primary temporal split, the classifier achieved an F1 of 0.620 and ROC-AUC of 0.777, indicating that the selected features carry meaningful predictive signal for this operationally defined target. Leave-One-Repository-Out evaluation, however, produced a substantially lower mean F1 of 0.315, showing that the learned patterns do not transfer uniformly to unseen repositories – most notably to Flask, the lowest-positive-rate repository in the dataset.

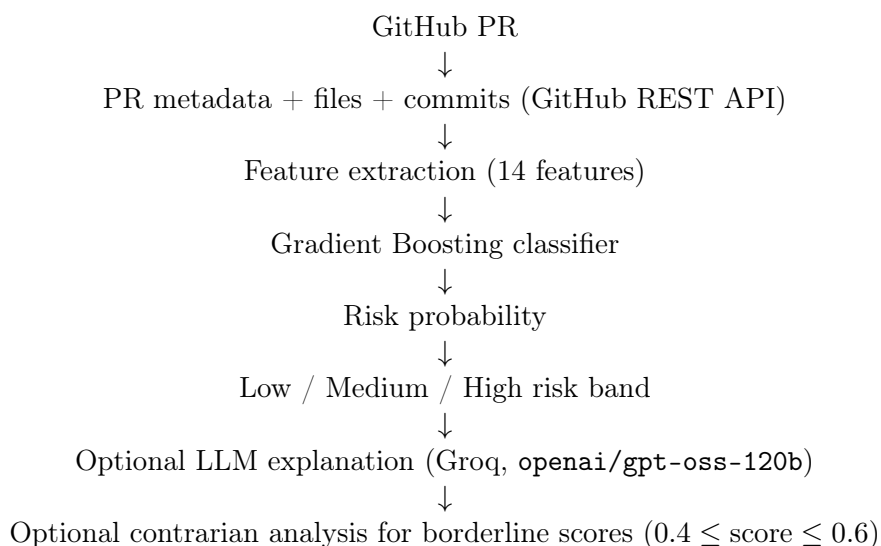
The central conclusion is not that pull-request bugs can be reliably predicted across arbitrary projects, but that repository history and change characteristics provide useful experimental signal for estimating a specific, narrowly defined bug-fix-association outcome within the repositories studied. The temporal-versus-LORO gap illustrates why both forms of validation matter in software engineering prediction research, and why single-split results should not be generalized to new projects without further evaluation.

The LLM layer successfully adds evidence-grounded qualitative explanation and a contrarian analysis for borderline predictions without altering the underlying machine learning score; its role is best understood as interpretability support, not as an independent source of predictive evidence.

Future work should expand the dataset across more and more diverse repositories, evaluate probability calibration directly, add established defect-prediction baselines, remove the bounded-file-search limitation on labeling and churn features, unify class-balancing treatment between the temporal and LORO training paths, and investigate features that generalize better across projects.

16 Reproducibility

The implementation is a Python project comprising source modules (`src/`), an automated test suite (`tests/`), a command-line entry point (`run.py`), and a standalone LORO evaluation script (`scripts/loro_eval.py`). The primary pipeline is:



The project supports dataset collection, model training (temporal evaluation printed as the primary result), real-PR scoring, synthetic-pipeline testing, and LORO evaluation via its CLI and scripts. For this paper, the automated test suite was executed directly and passed 37/37 tests, with 18 harmless NumPy/joblib deprecation warnings and no test failures; the primary temporal-split training path and the LORO script were executed directly against the shipped dataset (`data/processed/all_repos.csv`, 547 rows) to verify the reported metrics rather than transcribing them from prior documentation. Real-PR demonstration scores (Section 12.4) require a live GitHub token and Groq API key and were not independently re-verified for this paper; they are reported as exploratory, previously documented behavior rather than as re-confirmed benchmark numbers.

17 References

References

- [1] Just-in-Time Software Defect Prediction Techniques: A Survey. In: *2024 15th International Conference on Information and Communication Systems (ICICS)*, 2024. doi:10.1109/ICICS63486.2024.10638276.

- [2] F. Qiu, Z. Gao, X. Xia, D. Lo, J. Grundy, and X. Wang, “Deep Just-In-Time Defect Localization,” *IEEE Transactions on Software Engineering*, vol. 48, no. 12, pp. 5068–5086, 2022. doi:10.1109/TSE.2021.3135875.
- [3] C. Pornprasit, C. Tantithamthavorn, J. Jiarpakdee, M. Fu, and P. Thongtanunam, “PyExplainer: Explaining the Predictions of Just-in-Time Defect Models,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021, pp. 407–418. doi:10.1109/ASE51524.2021.9678763.
- [4] A. Collante, S. Abedu, S. Khatoonabadi, A. Abdellatif, E. Alor, and E. Shihab, “The Impact of Large Language Models (LLMs) on Code Review Process,” *arXiv preprint arXiv:2508.11034*, 2025. doi:10.48550/arXiv.2508.11034.
- [5] U. Cihan, V. Haratian, A. İçöz, M. K. Gül, Ö. Devran, E. F. Bayendur, B. M. Uçar, and E. Tüzün, “Automated Code Review in Practice,” in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 425–436. doi:10.1109/ICSE-SEIP66354.2025.00043.

A Appendix: System Architecture

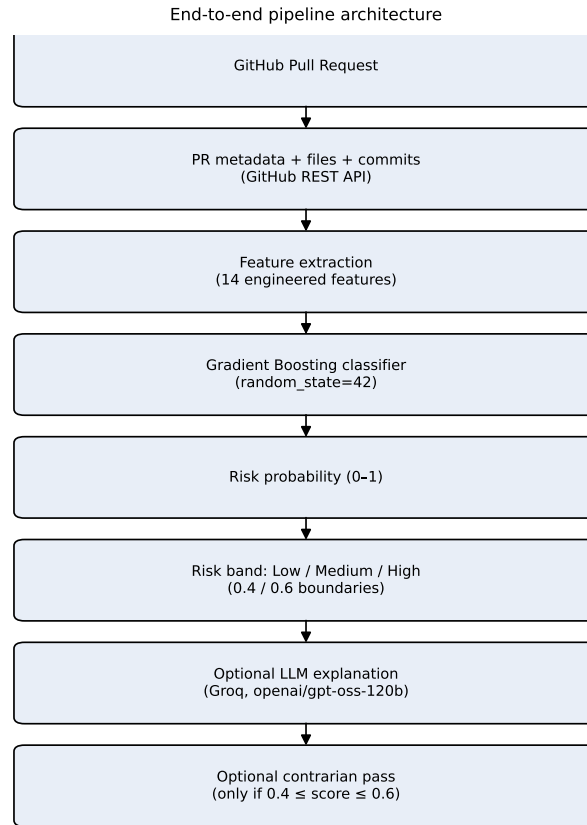


Figure 5: End-to-end pipeline architecture: GitHub PR ingestion, feature extraction, the Gradient Boosting classifier, risk banding, and the optional LLM explanation and contrarian stages.

B Appendix: Example Prediction Report Shape

```

{
  "repo": "pallets/flask",
  "pr_number": 123,
  "risk_score": 0.51,
  "risk_label": "medium",
  "explanation": null,
  "llm_status": "skipped",
  "contrarian": null
}

```

This shows the structure of a pipeline report; the numeric value is illustrative of *shape* only and is not a measured experimental result.